Державний вищий навчальний заклад
Ужгородський національний університет
Факультет інформаційних технологій

Лабораторна робота № 3

Тема: «Практикум із розроблення через тестування. Цикл Red - Green - Refactor»

Виконав студент IV курсу
Спеціальності «Інженерія
програмного забезпечення»
Заплетнюк Євгеній Сергійович

Ужгород-2026

1. Мета, завдання та інструменти

Я виконав лабораторну роботу з дисципліни «Автоматизація тестування», щоб відпрацювати розроблення через тестування. Мета - додавати поведінку невеликими кроками, спочатку бачити падіння тесту, а після реалізації безпечно змінювати структуру коду.

У показовій частині я реалізував тарифікатор паркування calcParkingFee. Індивідуальне завдання - варіант 2, калькулятор лікарняних calcSickPay. Для нього виконано 6 повних циклів Red - Green - Refactor з окремими комітами. Додатково я перевірив шість навмисних помилок і підготував CLI для запуску обох модулів.

Я продовжив наявний репозиторій Testing-Automation-University у гілці lab3-variant2, створеній від lab2-variant2. Попередні 74 тести залишилися в проєкті. Повний код, тести, журнали запусків та історію Git я здаю окремо; у звіті навів лише фрагменти, потрібні для пояснення.

Фактичні версії інструментів наведено в таблиці 1. Залежності відтворюються через package-lock.json, а Node.js виконує демонстраційні TypeScript-файли без окремого збирання.

Інструмент	Версія / призначення
ОС	Windows, версія ядра 10.0.26300, x64
Node.js / npm	24.21.0 / 11.19.0
TypeScript	6.0.3; strict: true
Vitest / @vitest/coverage-v8	5.0.3 / 5.0.3; модульні тести та покриття
ESLint / Prettier	10.12.0 / 3.9.10
Git	2.56.0.windows.2; окремі коміти кроків
Звіт	DOCX; Python 3.14.7, python-docx; перевірка у Microsoft Word

Таблиця 1 - Середовище виконання

2. Принципи TDD та перевірки

Я використовував тест як наступну вимогу до функції. На кроці Red потрібне падіння саме через відсутню поведінку. Помилка імпорту припустима лише на старті нового модуля. Далі тести повинні запускатися й показувати різницю між очікуваним та фактичним результатом.

Послідовність роботи подано в таблиці 2. Після кожного кроку я зберігав вивід Vitest у evidence/parking або evidence/sickPay та додавав його до

відповідного коміту. Такий журнал дає змогу перевірити результат запуску разом зі станом коду.

Крок	Що я робив	Умова переходу
Red	Додавав тест наступної вимоги	Новий тест падає з очікуваної причини
Green	Додавав код для поточної вимоги	Поточний набір модуля проходить
Refactor	Виділяв функції та константи	Тести проходять без зміни очікувань

Таблиця 2 - Правила циклу розроблення

Для числових порогів я перевіряв значення з обох боків межі. У калькуляторі лікарняних це 4/5/6, 9/10/11 і 29/30/31 днів. Параметризовані it.each зменшують повторення тестового коду, але очікувані суми в наборах залишаються явними числами.

Покриття показує, які ділянки виконалися під час тестів. Саме по собі воно не доводить правильність перевірок. Тому я додатково змінив ставки, межі й округлення та перевірив, чи набір помітить ці дефекти.

3. Специфікація та граничні випадки

3.1. Показовий модуль паркування

Я зберіг правила тарифікації з методички. Вхід minutes - невід’ємне ціле число хвилин. Необов’язкові прапорці night і weekend мають тип boolean. У таблиці 3 зафіксовано порядок операцій, бо перенесення знижки перед обмеженням змінює суму.

Правило	Поведінка
Валідація	Від’ємне, дробове, NaN та нескінченне значення - INVALID_DURATION
0-15 хвилин	0 грн
16-60 хвилин	30 грн
Понад 60 хвилин	Ще 20 грн за кожну розпочату годину
Обмеження	Базова сума не перевищує 200 грн
Ніч / вихідний	Після обмеження множення на 0,5, потім на 0,8
Результат	Число гривень, округлене до двох десяткових знаків

Таблиця 3 - Правила тарифікатора паркування

Як і приклад у методичці, функція застосовує одне обмеження 200 грн до переданого інтервалу. Поділ на кілька календарних діб не реалізовано. Для інтервалів понад добу потрібна окрема уточнена специфікація; перевірені контрольні приклади охоплюють 0-1440 хвилин.

3.2. Варіант 2 - калькулятор лікарняних

Сигнатура мого модуля - calcSickPay(days, dailyAverage). Я прийняв dailyAverage як середньоденний заробіток у гривнях. Ставка застосовується окремо до днів кожного періоду, а не до всіх днів за ставкою останнього періоду. Методичка не уточнює ці деталі, тому я явно зафіксував їх перед реалізацією.

Нуль днів і нульовий заробіток допустимі. Понад 30 днів я відхиляю, а не обрізаю до 30. Спочатку перевіряю дні, потім заробіток; некоректний заробіток відхиляється навіть для нуля днів. Це навчальна специфікація, без правил реальних виплат.

У таблиці 4 вимоги розкладено в порядку реалізації. Правила валідації та округлення уточнюють контракт варіанта, а ставки 60 %, 80 % і 100 % взято безпосередньо із завдання.

Цикл	Вимога	Перевірені межі
1	Цілі дні від 0 до 30; інакше INVALID_DAYS	-1; 0,5; 30,5; 31; NaN; ±Infinity
2	Скінченний невід'ємний dailyAverage; інакше INVALID_DAILY_AVERAGE	-0,01; 0; 0,01; NaN; ±Infinity
3	Дні 1-5: 60 % dailyAverage за день	0; 1; 4; 5 днів
4	Дні 6-10: 80 % dailyAverage за день	5; 6; 9; 10 днів
5	Дні 11-30: 100 % dailyAverage за день	10; 11; 29; 30; 31 день
6	Округлення загальної суми до копійок	100,01 та 0,01 грн/день

Таблиця 4 - Послідовність вимог варіанта 2

Проміжні внески я не округлюю. Наприклад, два дні за 0,01 грн дають 0,012 грн, після загального округлення - 0,01 грн. Округлення кожного дня окремо дало б 0,02 грн і порушило б обраний контракт. Обчислення використовують number, тому довільно великі суми й повна десяткова точність не гарантуються.

4. Конкретні розрахунки та приклади

Для паркування тривалістю 600 хвилин я отримую 9 розпочатих годин понад першу: 30 + 9 × 20 = 210 грн. Обмеження зменшує суму до 200 грн. Нічний тариф дає 100 грн, вихідний - 80 грн. Якщо нічний коефіцієнт застосувати до обмеження, вийшло б 105 грн перед знижкою вихідного.

Для лікарняного на 11 днів із середньоденним заробітком 333,33 грн я розбив дні на три періоди. Таблиця 5 показує проміжні внески. Загальна сума дорівнює 2666,64 грн; саме це значення перевіряють тест і CLI.

Період	Днів	Розрахунок, грн	Внесок, грн
1-5	5	5 × 333,33 × 0,6	999,99
6-10	5	5 × 333,33 × 0,8	1333,32
11	1	1 × 333,33 × 1	333,33
Разом	11	999,99 + 1333,32 + 333,33	2666,64

Таблиця 5 - Розрахунок лікарняних за трьома періодами

На межі 5 днів за 100 грн/день сума дорівнює 300 грн. Шостий день додає 80 грн, тому результат - 380 грн. Десять днів дають 700 грн, а одинадцятий додає вже 100 грн. Такі приклади відрізняють поетапне нарахування від помилкового застосування однієї ставки до всього періоду.

5. Структура програми та ключові рішення

Я відокремив чисті розрахунки від запуску команд. Тести звертаються до двох публічних функцій, а допоміжні функції залишаються всередині модулів. Призначення основних файлів наведено в таблиці 6.

Модуль	Що робить
src/parking.ts	Валідує хвилини, розраховує тариф і застосовує знижки
src/sickPay.ts	Перевіряє дні та заробіток, нараховує суму за періодами
tests/parking.test.ts	35 тестів показової частини
tests/sickPay.test.ts	35 тестів індивідуального варіанта
tools/cli.ts	Запускає розрахунок із командного рядка
tools/vectors.ts	Повторно обчислює 70 рядків таблиці звіту
vitest.config.ts	Налаштовує V8, звіти та пороги покриття
evidence/	Зберігає журнали кроків, коміти й фактичні результати

Таблиця 6 - Файли лабораторної роботи

У calcWeightedDays я обчислюю кількість днів із урахуванням коефіцієнтів. Перший період обмежений п’ятьма днями, другий також має щонайбільше п’ять. Для третього періоду залишаються дні понад десятий. Фрагмент у лістингу 1 є кінцевим кодом цієї функції.

```
function calcWeightedDays(days: number): number {
  const firstDays = Math.min(days, FIRST_PERIOD_DAYS);
  const secondDays = Math.min(
    Math.max(0, days - FIRST_PERIOD_DAYS),
    SECOND_PERIOD_END - FIRST_PERIOD_DAYS,
  );
  const remainingDays = Math.max(0, days - SECOND_PERIOD_END);
  return firstDays * FIRST_RATE + secondDays * SECOND_RATE + remainingDays;
}
```

Лістинг 1 - Зважені дні за періодами (sickPay.ts)

У публічній функції я залишив послідовність із трьох кроків: перевірити дні, перевірити заробіток, округлити загальну суму. Лістинг 2 показує цей контракт; помилки виникають до розрахунку.

```
function roundMoney(amount: number): number {
  return Math.round(amount * 100) / 100;
}
export function calcSickPay(days: number, dailyAverage: number): number {
  assertValidDays(days);
  assertValidDailyAverage(dailyAverage);
  return roundMoney(dailyAverage * calcWeightedDays(days));
}
```

Лістинг 2 - Округлення й публічна функція (sickPay.ts)

У тарифікаторі паркування обмеження належить базовому тарифу. Окрема applyRates отримує вже обмежену суму, тому нічна знижка працює і для першої години. Це видно в лістингу 3.

```
function applyRates(fee: number, options: ParkingOptions): number {
  let result = fee;
  if (options.night) result *= NIGHT_RATE;
  if (options.weekend) result *= WEEKEND_RATE;
  return result;
}
export function calcParkingFee(
  minutes: number,
  options: ParkingOptions = {},
): number {
  assertValidDuration(minutes);
  return Math.round(applyRates(calcBaseFee(minutes), options) * 100) / 100;
}
```

Лістинг 3 - Знижки після базового тарифу (parking.ts)

Після четвертого циклу лікарняних я замінив довгий вираз окремими firstDays і secondDays. Після п’ятого виділив calcWeightedDays, додав назви SECOND_PERIOD_END та MAX_DAYS. Тести не змінювали очікувань і залишалися зеленими. Після шостого циклу виніс округлення в roundMoney.

В індивідуальному модулі виконано 6 рефакторингів, у паркуванні - 2. На жодному з цих восьми кроків тести не зламалися. Повний журнал містить 13 кроків Red, 13 Green та 8 Refactor; окремий коміт фіксує приклади округлення паркування, які вже проходили.

6. Контрольні вектори з завдання

У таблиці 7 я звів характерні приклади паркування з методички та ручні розрахунки за правилами варіанта 2. У методичці немає окремої таблиці числових векторів лікарняних, тому очікувані суми для них я розрахував за ставками 60 %, 80 % і 100 %. Фактичні значення отримано виконанням коду.

Вхід	Очікувано	Фактично	Статус
Паркування: -1 хв	INVALID_DURATION	INVALID_DURATION	Пройдено
15 хв	0	0	Пройдено
16 / 60 хв	30 / 30	30 / 30	Пройдено
61 / 120 / 121 хв	50 / 50 / 70	50 / 50 / 70	Пройдено
540 / 600 / 1440 хв	190 / 200 / 200	190 / 200 / 200	Пройдено
600 хв; night	100	100	Пройдено
600 хв; weekend	160	160	Пройдено
600 хв; обидва прапорці	80	80	Пройдено
Лікарняні: 5 дн.; 100 грн	300	300	Пройдено
6 / 10 / 11 дн.; 100 грн	380 / 700 / 800	380 / 700 / 800	Пройдено
30 дн.; 100 грн	2700	2700	Пройдено
31 дн.; 100 грн	INVALID_DAYS	INVALID_DAYS	Пройдено

Таблиця 7 - Контрольні приклади та результати

7. Тести та фактичне покриття

Я використав явні очікувані значення та параметризовані тести. Для паркування є 7 блоків it.each і один звичайний тест, для лікарняних - 6 блоків it.each та один звичайний тест. У сумі ці блоки утворюють 70 тестів, по 35 на модуль.

Таблиця 8 містить усі нові тестові випадки. Одиниця числового результату - гривня. У рядках із помилкою фактичне значення - повідомлення винятку. Для компактності позначення E1 означає INVALID_DURATION, E2 - INVALID_DAYS, E3 - INVALID_DAILY_AVERAGE.

№	Група	Вхідні дані	Очікувано	Фактично	Статус
1	П: валідація	-1 хв	E1	E1	Так
2	П: валідація	-60 хв	E1	E1	Так
3	П: валідація	10.5 хв	E1	E1	Так
4	П: валідація	NaN хв	E1	E1	Так
5	П: валідація	∞ хв	E1	E1	Так
6	П: безкоштовно	0 хв	0	0	Так
7	П: безкоштовно	1 хв	0	0	Так
8	П: безкоштовно	14 хв	0	0	Так
9	П: безкоштовно	15 хв	0	0	Так
10	П: тариф	16 хв	30	30	Так
11	П: тариф	59 хв	30	30	Так
12	П: тариф	60 хв	30	30	Так
13	П: тариф	61 хв	50	50	Так
14	П: тариф	119 хв	50	50	Так
15	П: тариф	120 хв	50	50	Так
16	П: тариф	121 хв	70	70	Так
17	П: тариф	180 хв	70	70	Так
18	П: тариф	539 хв	190	190	Так
19	П: тариф	540 хв	190	190	Так
20	П: тариф	541 хв	200	200	Так
21	П: тариф	600 хв	200	200	Так
22	П: тариф	1440 хв	200	200	Так
23	П: ніч	0 хв; night	0	0	Так
24	П: ніч	15 хв; night	0	0	Так

№	Група	Вхідні дані	Очікувано	Фактично	Статус
25	П: ніч	16 хв; night	15	15	Так
26	П: ніч	60 хв; night	15	15	Так
27	П: ніч	121 хв; night	35	35	Так
28	П: ніч	600 хв; night	100	100	Так
29	П: вихідний	15 хв; night=false	0	0	Так
30	П: вихідний	16 хв; night=false	24	24	Так
31	П: вихідний	600 хв; night=false	160	160	Так
32	П: вихідний	600 хв; night=true	80	80	Так
33	П: вихідний	45 хв; night=true	12	12	Так
34	П: вихідний	121 хв; night=true	28	28	Так
35	П: округлення	45 / 121 хв; обидва прапорці	12 / 28	12 / 28	Так
36	Л: дні	-1 дн.; 100 грн/дн.	E2	E2	Так
37	Л: дні	0.5 дн.; 100 грн/дн.	E2	E2	Так
38	Л: дні	30.5 дн.; 100 грн/дн.	E2	E2	Так
39	Л: дні	31 дн.; 100 грн/дн.	E2	E2	Так
40	Л: дні	NaN дн.; 100 грн/дн.	E2	E2	Так
41	Л: дні	∞ дн.; 100 грн/дн.	E2	E2	Так
42	Л: дні	-∞ дн.; 100 грн/дн.	E2	E2	Так
43	Л: заробіток	1 дн.; -0.01 грн/дн.	E3	E3	Так
44	Л: заробіток	1 дн.; -100 грн/дн.	E3	E3	Так
45	Л: заробіток	1 дн.; NaN грн/дн.	E3	E3	Так
46	Л: заробіток	1 дн.; ∞ грн/дн.	E3	E3	Так
47	Л: заробіток	1 дн.; -∞ грн/дн.	E3	E3	Так
48	Л: порядок перевірок	0 дн.; -1 грн/дн.	E3	E3	Так
49	Л: 60 %	0 дн.; 100 грн/дн.	0	0	Так
50	Л: 60 %	1 дн.; 100 грн/дн.	60	60	Так
51	Л: 60 %	4 дн.; 100 грн/дн.	240	240	Так
52	Л: 60 %	5 дн.; 100 грн/дн.	300	300	Так
53	Л: 60 %	5 дн.; 0 грн/дн.	0	0	Так
54	Л: 60 %	1 дн.; 1000 грн/дн.	600	600	Так
55	Л: періоди	6 дн.; 100 грн/дн.	380	380	Так
56	Л: періоди	9 дн.; 100 грн/дн.	620	620	Так
57	Л: періоди	10 дн.; 100 грн/дн.	700	700	Так
58	Л: періоди	11 дн.; 100 грн/дн.	800	800	Так
59	Л: періоди	12 дн.; 100 грн/дн.	900	900	Так
60	Л: періоди	29 дн.; 100 грн/дн.	2600	2600	Так
61	Л: періоди	30 дн.; 100 грн/дн.	2700	2700	Так
62	Л: округлення	1 дн.; 100.01 грн/дн.	60.01	60.01	Так
63	Л: округлення	5 дн.; 100.01 грн/дн.	300.03	300.03	Так
64	Л: округлення	6 дн.; 100.01 грн/дн.	380.04	380.04	Так
65	Л: округлення	10 дн.; 100.01 грн/дн.	700.07	700.07	Так
66	Л: округлення	11 дн.; 100.01 грн/дн.	800.08	800.08	Так
67	Л: округлення	30 дн.; 100.01 грн/дн.	2700.27	2700.27	Так
68	Л: округлення	1 дн.; 0.01 грн/дн.	0.01	0.01	Так
69	Л: округлення	2 дн.; 0.01 грн/дн.	0.01	0.01	Так
70	Л: округлення	11 дн.; 333.33 грн/дн.	2666.64	2666.64	Так

Таблиця 8 - Повний набір тестів лабораторної 3

Підсумок запуску: 70 із 70 нових тестів пройдено. Разом із попередніми роботами пройдено 144 із 144 тестів у 6 файлах, пропущених і невдалих тестів немає. Повний запуск із V8 тривав 272 мс за показником Duration у Vitest. Час залежить від середовища; журнал збережено в evidence/coverage.txt.

Покриття нових модулів наведено в таблиці 9. Я не додавав тестів лише заради відсотка: пороги вже досягнуто тестами поведінки. CLI та генератор

таблиці не входять до показника покриття бізнес-логіки; для CLI окремо виконано сім контрольних запусків.

Файл	Рядки	Гілки	Функції	Оператори
parking.ts	18/18 (100 %)	11/11 (100 %)	4/4 (100 %)	21/21 (100 %)
sickPay.ts	17/17 (100 %)	9/9 (100 %)	5/5 (100 %)	17/17 (100 %)

Таблиця 9 - Покриття нових модулів засобами V8

У parking.ts є 36 фізичних рядків, із них 18 V8 враховує як виконувані. У sickPay.ts - 30 фізичних рядків і 17 виконуваних. Непокритих виконуваних рядків в обох модулях - 0. Порожні рядки, типи й окремі структурні рядки не слід змішувати з виконуваними рядками звіту покриття.

Перевірка типів, ESLint і Prettier завершилися з кодом 0. Для Prettier я задав endOfLine: auto, щоб він приймав наявні переноси рядків Windows. Журнали evidence/typecheck.txt, evidence/lint.txt і evidence/format.txt збережено поряд із результатами тестів.

8. Власні вхідні дані та результати

Я перевірив власний приклад: 11 днів і 333,33 грн середньоденного заробітку. Результат - 2666,64 грн, що збігається з проміжними сумами таблиці 5. Другий приклад - 6 днів за 100,01 грн: без округлення виходить 380,038 грн, функція повертає 380,04 грн.

Лістинг 4 містить фактичні команди й відповіді. Для 31 дня CLI повертає повідомлення INVALID_DAYS і код завершення 1, а не суму за 30 днів. Для правильних даних код завершення дорівнює 0.

```
> node tools/cli.ts sick 11 333.33
Sick pay: 2666.64 UAH
> node tools/cli.ts sick 6 100.01
Sick pay: 380.04 UAH
> node tools/cli.ts sick 31 100
INVALID_DAYS
> node tools/cli.ts parking 600 --night --weekend
Parking fee: 80.00 UAH
```

Лістинг 4 - Власні приклади та відповіді програми (cli.ts)

Текстові результати форматуються через toFixed(2), тому в CLI видно дві цифри після крапки. Функції повертають number: значення 80 і 80,00 для цього типу однакові. Розшифрування даних у цій роботі відсутнє, оскільки завдання стосується тарифів і тестування, а не шифрування.

9. Інтерфейс і запуск програми

9.1. Командний інтерфейс

Я запускаю команди з кореня репозиторію. Після отримання гілки lab3-variant2 достатньо виконати npm ci. Команда test:lab3 перевіряє лише нові модулі, а coverage запускає всі тести, включно з попередніми роботами. Послідовність наведено в лістингу 5.

```
git switch lab3-variant2
npm ci
npm run test:lab3
npm run coverage
npm run typecheck
npm run lint
npm run format:check
npm run demo -- sick 11 333.33
npm run demo -- parking 600 --night --weekend
```

Лістинг 5 - Встановлення, перевірки та запуск (package.json)

На рисунку 1 показано HTML-перегляд збереженого журналу семи справжніх CLI-запусків. Це читабельне відображення evidence/cli.txt, а не окремий інтерфейс калькулятора. Поряд із відповіддю наведено код завершення кожної команди.

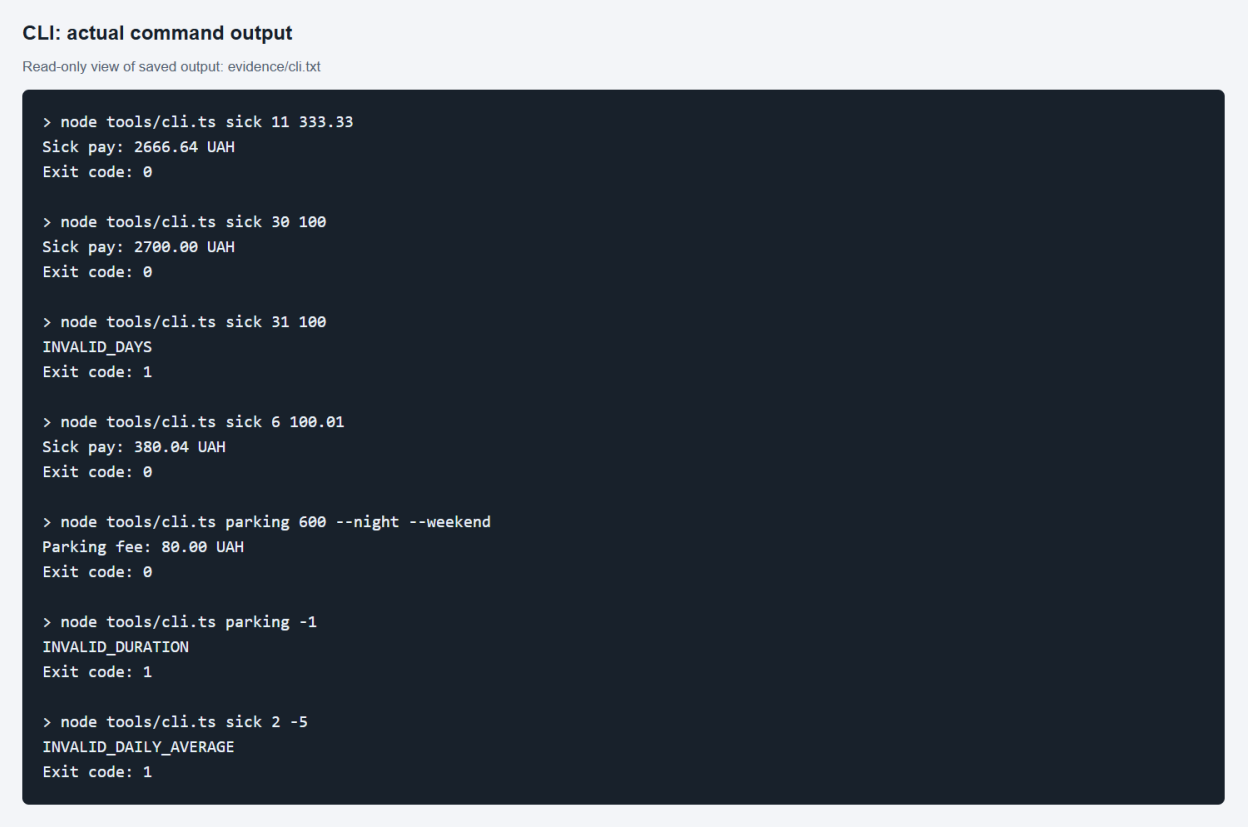


Рисунок 1 - Журнал CLI із сумами та повідомленнями помилок

9.2. Графічний перегляд результатів

Окреме GUI калькулятора методичка не вимагає, тому я його не додавав. Для графічного аналізу використав HTML-звіт V8: після npm run coverage відкриваю coverage/index.html у браузері. Рисунок 2 показує фактичне покриття всіх модулів репозиторію.

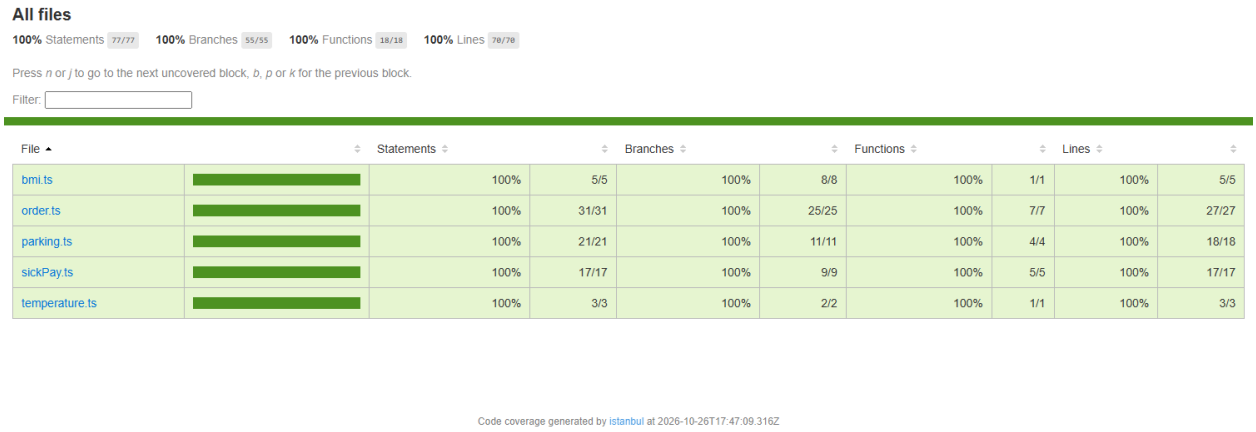


Рисунок 2 - HTML-звіт V8 із покриттям 100 %

Натискання sickPay.ts відкриває сторінку з кодом і лічильниками виконання. У таблиці звіту всі 9 гілок цього модуля покриті. Сам HTML-звіт включено до архіву, тому результати можна переглянути й без повторного встановлення залежностей.

10. Додаткові перевірки та матеріали подання

10.1. Внесення контрольованих помилок

Я тимчасово вносив по одній помилці й запускав тести відповідного модуля. Після кожного запуску відновлював початковий файл. Таблиця 10 показує фактичну кількість тестів, що впали. Усі шість змін виявлено; після відновлення коду знову пройшли 144 тести.

Зміна	Тестів упало	Результат
30 днів помилково відхиляються	2	Виявлено
Змінено ставку першого періоду	18	Виявлено
Змінено ставку другого періоду	12	Виявлено
Прибрано округлення суми	5	Виявлено
15 хвилин помилково платні	3	Виявлено
Завищено добову межу	6	Виявлено

Таблиця 10 - Результати контрольованих мутацій

Ця перевірка показала, що тести реагують на зміну поведінки, а не лише виконують рядки. Наприклад, вилучення округлення зламало 5 тестів, а

помилкове відхилення 30 днів - 2. Це невеликий ручний набір мутацій, а не повний автоматичний mutation score.

10.2. Історія Git і матеріали репозиторію

Репозиторій роботи: https://github.com/YevgeniyZapletnyk/Testing-Automation-University. Гілка лабораторної - lab3-variant2, базовий коміт lab2-variant2 - a56e6a6. Повну історію нової роботи можна отримати командою git log --reverse --oneline origin/lab2-variant2..lab3-variant2. Її вивід збережено в evidence/git-log.txt.

У таблиці 11 я навів коміти шести завершених циклів індивідуального варіанта. Кожен рядок має окремі Red, Green і Refactor; це дає змогу відкрити стан до реалізації, після реалізації та після зміни структури.

Цикл	Red	Green	Refactor	Що змінено
1	049d18b	94fdab9	db27a16	окрема перевірка днів
2	1466fc7	f9c641b	0bfbcbb	окрема перевірка заробітку
3	da7297c	a44bcc1	8c240e1	константи першого періоду
4	a7e449e	54b479d	a587dd9	окремі лічильники оплачуваних днів
5	e7cbf1d	795215a	64f67a4	функція зважених днів і межі періодів
6	0c3a225	134093d	49cfa7c	іменована функція округлення

Таблиця 11 - Коміти циклів калькулятора лікарняних

У лістингу 6 наведено фактичний фрагмент git log для індивідуального модуля. Журнали кроків дозволяють перевірити, що Red завершувався помилкою, а Green і Refactor - успіхом. Коміт на кроці Red не містить реалізації нової вимоги.

```
049d18b red: sickPay 1 - діапазон цілих днів 0-30
94fdab9 green: sickPay 1 - діапазон цілих днів 0-30
db27a16 refactor: sickPay 1 - окрема перевірка днів
1466fc7 red: sickPay 2 - коректний середньоденний заробіток
f9c641b green: sickPay 2 - коректний середньоденний заробіток
0bfbcbb refactor: sickPay 2 - окрема перевірка заробітку
da7297c red: sickPay 3 - 60 відсотків за дні 1-5
a44bcc1 green: sickPay 3 - 60 відсотків за дні 1-5
8c240e1 refactor: sickPay 3 - константи першого періоду
a7e449e red: sickPay 4 - 80 відсотків за дні 6-10
54b479d green: sickPay 4 - 80 відсотків за дні 6-10
a587dd9 refactor: sickPay 4 - окремі лічильники оплачуваних днів
e7cbf1d red: sickPay 5 - повна оплата днів 11-30
795215a green: sickPay 5 - повна оплата днів 11-30
64f67a4 refactor: sickPay 5 - функція зважених днів і межі періодів
0c3a225 red: sickPay 6 - округлення загальної суми до копійок
134093d green: sickPay 6 - округлення загальної суми до копійок
49cfa7c refactor: sickPay 6 - іменована функція округлення
```

Лістинг 6 - Шість циклів у порядку створення (git-log.txt)

У показовій частині приклади округлення 45 хвилин → 12 грн і 121 хвилина → 28 грн пройшли до додавання Math.round. Я не називав цей запуск Red: він записаний окремим тестовим комітом. На поточних тарифах результат і так має точність до копійок. У варіанті 2 тести дробового заробітку справді падали без округлення.

10.3. Рецензія на приклад однокурсника

Я переглянув наданий звіт Теліги Олександра Михайловича, варіант 1 - тарифікатор доставки. Підстава цієї рецензії - текст документа, лістинги й знімки історії. Репозиторій за URL із документа під час перевірки відповів Repository not found, тому я не стверджую, що перевірив вміст кожного його коміту.

На знімках історії Red передує Green, а між реалізаціями є Refactor. Видимих випадків, де нова реалізація передує відповідному тестовому коміту, я не знайшов. Водночас за назвами комітів неможливо довести їхній повний зміст. Окремо колега пояснив, що тест округлення паркування вже був зеленим; такий крок не варто рахувати як підтверджений Red.

За наведеним описом мінімальний Green спочатку завжди кидав помилку ваги, потім повертав 50 грн, а розпочаті кілограми додавалися лише після наступного тесту. Це відповідає логіці малих кроків. Я б перевірив цю послідовність за diff комітів, коли репозиторій стане доступним.

Рефакторинги у звіті предметні: магічні числа замінено константами, валідацію та ваговий тариф розділено, округлення ваги винесено в roundToGrams. Приклад 0,2 + 4,9 + 0,9 кг пояснює потребу в точності до грама. У текстовому лістингу DeliveryOptions є описка «oolean»; її потрібно виправити на boolean перед копіюванням коду. Загалом документ добре пояснює порядок правил, але недоступний репозиторій обмежує перевірку історії.

Висновки

Я реалізував тарифікатор паркування та калькулятор лікарняних для варіанта 2. Паркування враховує безкоштовні хвилини, погодинний тариф, обмеження й дві знижки. Калькулятор лікарняних окремо нараховує оплату за три періоди та округлює загальну суму.

Для індивідуального модуля я виконав 6 повних циклів Red - Green - Refactor. Кожен крок має свій коміт і журнал запуску. Найбільше уваги потребували межі 5, 10 і 30 днів та вибір моменту округлення. Я зафіксував ці рішення тестами до відповідної реалізації.

Усі 70 нових тестів і 74 тести попередніх робіт пройшли. Нові модулі мають 100 % покриття гілок, а вісім рефакторингів не змінили результатів. Шість навмисних помилок набір виявив, зокрема неправильну ставку та вилучення округлення.

Після першого кроку хотілося одразу написати всі тарифні правила, бо вони вже були відомі. Я зберіг малий обсяг кожної реалізації: на ранніх кроках функція ще повертала нуль для прийнятих даних, а нові формули з’являлися після тестів. На цій задачі такий порядок дав зрозумілу історію та спростив виділення допоміжних функцій.

Рішення має визначені межі: середньоденний заробіток задається готовим числом, лікарняний понад 30 днів відхиляється, а паркування не розбивається на календарні доби. Тип number не забезпечує довільної десяткової точності. Для навчальних контрольних прикладів поведінка перевірена; розширення правил потребуватиме нових тестів і уточнення специфікації.

Відповіді на контрольні запитання

1. Назвіть три кроки циклу TDD і сформулюйте правило для кожного.

Red - я додаю тест наступної вимоги й перевіряю його падіння. Green - додаю мінімальний код, який робить поточні тести зеленими. Refactor - змінюю структуру коду без зміни поведінки та повторно запускаю тести.

2. Чому важливо побачити, як тест падає, перш ніж писати код? Які дві причини можуть зробити щойно написаний тест зеленим?

Падіння підтверджує, що тест здатний помітити відсутню поведінку. Якщо він одразу зелений, поведінка або вже реалізована, або тест не перевіряє потрібний результат. У моїй показовій частині приклади округлення вже проходили; це перший випадок. Відсутній expect чи пропущений await можуть створити другий.

3. Чому на кроці Green пишуть мінімальний код? Наведіть приклад із власної роботи.

Я обмежую реалізацію поточною вимогою, щоб не додавати неперевірені правила наперед. Після перевірки днів calcSickPay ще повертав нуль для прийнятих значень. Формула 60 % з'явилася після тесту на один оплачуваний день, а правило 80 % - після тесту на шостий день.

4. Чому рефакторинг виконують лише на зеленому наборі тестів?

Зелений набір дає вихідну точку для порівняння. Якщо після зміни структури тест падає, я можу пов'язати проблему з цією зміною. На вже червоному наборі важче відрізнити попередній дефект від нової помилки рефакторингу.

5. У якому циклі показової частини знадобився рефакторинг і що саме було покращено?

Після четвертого циклу я назвав тарифні константи замість чисел у формулі. Після перевірки восьмого пункту виділив assertValidDuration, calcBaseFee та applyRates. Так валідація, базовий тариф і знижки мають окремі відповідальності.

6. Поясніть, чому порядок застосування нічного тарифу й обмеження добової плати впливає на результат.

Для 600 хвилин базовий тариф без обмеження дорівнює 210 грн. Правильний порядок дає min(210, 200) × 0,5 = 100 грн. Якщо спочатку застосувати нічний коефіцієнт, буде min(105, 200) = 105 грн. Тест на 600 хвилин уночі фіксує результат 100 грн.

7. Що означає твердження «TDD дає зворотний зв’язок про архітектуру»? Чи відчули ви це під час роботи?

Тест змушує рано визначити вхід, вихід і відповідальність модуля. Я залишив у публічній calcSickPay лише перевірки та виклик розрахунку, а поділ днів виніс у calcWeightedDays. Це спростило код без залежності від CLI або способу відображення суми.

8. Чому робота, подана одним комітом, не зараховується, навіть якщо код і тести якісні?

Один готовий стан не показує порядку створення. За ним неможливо встановити, чи тест був написаний раніше, чи падав він і коли виконувався рефакторинг. Для цієї лабораторної процес є частиною результату, тому я зберіг окремі коміти та журнали.

9. Наведіть три класи задач, для яких TDD погано підходить.

Я б обережно застосовував TDD до дослідницьких прототипів із невідомим результатом, оцінювання зовнішнього вигляду інтерфейсу та первинного вивчення незнайомого API. У таких задачах спочатку часто потрібен експеримент або візуальна перевірка. Після уточнення поведінки автоматичні тести все одно корисні.

10. Яким є найкращий сценарій застосування TDD у промисловій роботі і чому?

Найзручніший сценарій - бізнес-логіка з чіткими правилами та числовими межами: тарифи, знижки, комісії, статуси. Я можу записати конкретний вхід і очікуваний вихід, а потім додавати правила по одному. Для виправлення дефекту підхід теж корисний: спочатку відтворювальний тест, потім зміна коду.

Джерела

1. Лабораторна 3_Практикум TDD.docx. Методичні вказівки з дисципліни «Автоматизація тестування». Наданий файл.

2. Теліга Олександр Михайлович. Лабораторна робота № 3, варіант 1. Наданий документ Лабораторна_робота_№_3_Виконав_Теліга_Олександр_Михайлович (4).docx. Використано для рецензії та зіставлення структури.

3. Vitest. Coverage. https://vitest.dev/guide/coverage.html. Дата звернення: 26.10.2026.

4. Vitest. Test API. https://vitest.dev/api/test. Дата звернення: 26.10.2026.

5. Fowler M. Test Driven Development. https://martinfowler.com/bliki/TestDrivenDevelopment.html. Дата звернення: 26.10.2026.

6. Заплетнюк Євгеній Сергійович. Testing-Automation-University, гілка lab3-variant2. https://github.com/YevgeniyZapletnyk/Testing-Automation-University. Код і журнали виконання лабораторної 3.